

NACRE: Rethinking Confidential Containers through Native Architectural Support

Linke Song^{*†}, Wenhao Wang^{*†✉}, Weijie Liu[‡], Rui Hou^{*†}

^{*}State Key Laboratory of Cyber Space Security, Defense Institute of Information Engineering, CAS, Beijing, China

[†]School of Cybersecurity, University of Chinese Academy of Sciences, Beijing, China

[‡]Nankai University

Abstract—Linux containers achieve high density and fast lifecycle operations by sharing the host kernel, but this design also lets a compromised host inspect or modify container state. Existing confidential-computing systems protect an enclave address space or an entire guest operating system, while recent container-granularity systems still add a separate protection context. These abstractions do not make a dynamic group of host-managed Linux processes the architectural protection unit.

This paper presents NACRE, a RISC-V hardware-software co-design for native confidential containers. Its key insight is to separate the host’s authority to manage resources from its authority to access or commit protected state. Hardware-recognized container identities direct protected traps to an isolated S-mode agent, while an M-mode monitor commits security-sensitive identity, mapping, and page transitions. The agent delegates services to host Linux without changing `satp`; services that neither access private bytes nor modify protected state also avoid M-mode. We prototype NACRE by extending QEMU, OpenSBI, Linux, a trusted agent, and `runc`. The prototype implements the single-container private-memory substrate and covered launch, fault, fork/COW, user-access, and teardown paths. Across five `lmbench` syscall and pipe metrics, the three-run means remain within 3.5% of the `runc-origin` baseline. With the eight `nginx` object-size means weighted equally, aggregate throughput is 1.9% lower.

I. INTRODUCTION

Linux containers have become a fundamental abstraction for deploying modern workloads. By combining namespaces, control groups, and filesystem isolation while sharing the host kernel, containers avoid virtual hardware and a separate guest operating system, thereby providing low startup latency, a small memory footprint, and efficient resource sharing [29], [31]. These properties have made containers widely used in cloud-native services, edge platforms, and high-performance computing environments [12], [13], [24]. More recently, AI-agent systems have begun launching containers on demand as sandboxes for tool invocation and generated-code execution. For example, LangGraph deploys self-hosted agent servers through container images and orchestration platforms [25], while OpenHands isolates agent-generated commands in Docker containers [32]. These emerging workloads further increase the demand for execution environments that combine strong isolation with the efficiency and flexibility of native containers.

The efficiency of containers, however, relies on a strong security assumption: the host operating system is trusted. Namespaces and control groups restrict the visibility and resource usage of container processes, but their enforcement ultimately depends on the host kernel. Once the host kernel

or a privileged administrator is compromised, the attacker can inspect container memory, modify page mappings and execution state, forge kernel-maintained container metadata, or access sensitive data such as credentials and user inputs. Consequently, conventional containers cannot protect workloads deployed on infrastructure whose privileged software is not fully trusted.

In this work, we define a *native confidential container* as a hardware-protected, dynamically managed group of Linux processes that continues to use host-native scheduling and resource management without relying on the host OS for the confidentiality or integrity of its execution state. The host may decide when processes run and which physical resources are allocated, but it must not be able to access protected container state, forge container membership, or convert resource-management authority into permission to read or modify protected resources. Realizing this abstraction requires three properties. First, the container must itself be the protection and lifecycle unit. Processes should be able to join and leave a protected container through operations such as `fork`, `clone`, `exec`, and `exit`, without reconstructing the protection boundary. Second, applications must retain native Linux process semantics, including copy-on-write memory, shared memory, signals, and synchronization among mutually trusting processes, without requiring source-code modification or a replacement programming model. Third, the host must continue to perform ordinary scheduling and resource-management functions, while access control and security-sensitive state transitions are enforced independently of the untrusted host. In particular, frequent container operations should not require switching into a VM, changing to a separate protected address space, or repeatedly invoking a high-privilege security monitor.

Existing confidential-computing abstractions do not simultaneously provide these properties. Enclave-based systems protect an isolated address space or a designated protected memory region [5], [9], [17], [26], [39]. Enclave runtimes can improve application compatibility, but the protection boundary remains different from a dynamically changing container process group. Supporting multi-process applications therefore requires reconstructing Linux process abstractions inside a library OS or runtime, or coordinating process and resource sharing across protected execution contexts. This complicates dynamic process management and weakens compatibility with native Linux semantics.

Confidential virtual machines take the opposite approach by protecting an entire guest OS using hardware mechanisms such as AMD SEV, Intel TDX, and Arm CCA [4], [21], [23], [38]. VM-backed container systems, including Kata Containers and Confidential Containers, preserve Linux compatibility by placing containers together with a guest kernel and supporting services inside a protected VM [14], [15], [35]. However, the VM rather than the container becomes the hardware protection and resource-management unit. Container creation, memory allocation, process management, and I/O are consequently distributed across the guest kernel, virtual machine monitor, and host. The additional guest software stack also reduces the density and responsiveness that motivate native containers.

Recent container-granularity approaches attempt to bridge the gap between enclaves and CVMs by moving protection closer to container processes while retaining selected host services [20], [43], [44], [46], [48]. However, these systems still introduce additional protection contexts or virtualization layers to mediate host interactions. As a result, they either sacrifice native Linux semantics or incur non-negligible overhead in frequent container operations. The container itself is not a hardware-recognized execution identity that can directly govern process membership, resource ownership, and security-event delivery while coexisting with the host’s native resource-management path.

The fundamental difficulty is that conventional hardware conflates two roles of the host OS: managing resources and serving as the security authority for those resources. Native containers require the host to retain the former role for efficiency, but an untrusted host cannot safely retain the latter. This tension is particularly visible on interrupts and exceptions. Delivering a confidential container’s events directly to the host exposes its architectural state to untrusted privileged software. Delivering every event to a high-privilege security monitor avoids this exposure, but places frequent system interactions on a costly path involving additional privilege transitions and protection-context changes. Native confidential containers therefore require both a hardware-recognized container identity and a protected mediation path that does not place a high-privilege monitor on every common operation.

Based on this observation, we present NACRE, a RISC-V hardware-software co-design architecture for native confidential containers. NACRE introduces *container identity* as architectural security state. Hardware associates execution contexts and protected resources with a container identity and performs access checks and event routing using this identity rather than host-maintained process metadata. Trusted mechanisms control the creation, inheritance, and revocation of identities, preventing the host from forging container membership or obtaining access to protected resources through page remapping.

NACRE further separates frequent mediation from security-critical enforcement through two isolated components. A lightweight *trusted container agent* executes at the same architectural privilege level as the host kernel but resides in a distinct hardware protection domain. Identity-aware event routing

delivers interrupts and exceptions originating from confidential containers to this agent, preventing the untrusted host from becoming the first recipient of protected architectural state. The agent protects container context and mediates interactions with host-provided Linux services, while events from ordinary host processes continue to follow the conventional Linux path. A minimal *security monitor* executes at a higher security level and handles only security-critical operations, such as container-identity management and protected-resource ownership transitions.

This separation allows frequent container-host interactions to remain in the native address-space and privilege context. The delegated trap path does not require entering a VM, switching page tables, or flushing the TLB; when a service neither accesses private bytes nor changes protected state, it also avoids the high-privilege monitor. The host can therefore continue to provide scheduling, physical-memory allocation, and ordinary Linux services, while hardware-enforced container identity prevents it from directly accessing protected state or fabricating security-relevant ownership. Dynamic process operations and inter-process sharing are expressed as changes within the same container identity rather than as coordination across independent protection contexts.

We implement a functional prototype spanning QEMU, OpenSBI, Linux, a trusted agent, and *runc*. It integrates registration and *exec*, demand paging, protected PTE updates, eager *fork* into fresh child pages, copy-on-write, typed kernel access to private user buffers, and teardown with the ordinary Linux lifecycle. The evaluated scope is the single-container private-memory substrate; cross-container ownership, protected shared objects, and attestation remain design-level mechanisms in this version.

QEMU-based functional tests exercise the covered launch and memory paths, and a dedicated parent/child workload validates the implemented fork transaction. For five *lmbench* syscall and pipe metrics, the three-run means are within 3.5% of the same-daemon *runc-origin* arm. With the eight *nginx* object-size means weighted equally, aggregate throughput is 1.9% lower. These results establish the feasibility of the native execution path and characterize its observed end-to-end behavior in QEMU.

Contributions. This paper makes the following contributions:

- We formulate the abstraction of native confidential containers and identify the fundamental conflict between untrusted-host isolation and host-native resource management. We derive the requirements for making a dynamic Linux process group the hardware protection and lifecycle unit while preserving native multi-process semantics.
- We design NACRE, a RISC-V hardware-software co-design architecture that introduces hardware-recognized container identities, identity-aware resource protection, and identity-aware event routing. NACRE separates a peer-privilege trusted container agent for frequent mediation from a high-privilege security monitor that commits security-sensitive state; delegated services that access neither private bytes

nor protected state avoid heavyweight protection-context transitions.

- We implement a QEMU-based prototype across OpenSBI, Linux, a trusted agent, and `runc`. The prototype preserves the covered native launch, private-memory, `fork`/COW, and teardown paths, and our preliminary evaluation characterizes common service paths and container workloads.

II. BACKGROUND & MOTIVATIONS

A. Native Container Execution Model

Linux containers isolate workloads through namespaces, control groups, and filesystem views while sharing the host kernel [29], [31]. After the container runtime establishes its namespaces, resource limits, filesystem view, and initial process, its tasks execute as ordinary Linux processes scheduled and managed by the host kernel. Accordingly, executing the same OCI image inside a protected VM or runtime does not by itself preserve the native container execution model. In this work, *native* means that host Linux remains responsible for ordinary process and resource management, while hardware and trusted software independently protect security-sensitive container state. This model has the following three properties.

Host-resident process lifecycle. A container comprises a dynamically changing group of host-scheduled Linux processes rather than a single address space or a guest operating system. Operations such as `fork`, `clone`, `exec`, and `exit` directly create, transform, and terminate processes within this group. Preserving this model requires that container processes remain directly visible to the host scheduler and process management subsystem, rather than being recreated through an additional guest OS or isolated runtime layer.

Host-resident resource management. Container resources follow the same lifecycle as ordinary Linux processes. For example, `brk` and `mmap` create and modify virtual memory regions, page faults trigger on-demand physical page allocation, and `fork` establishes copy-on-write relationships between parent and child processes. Linux maintains page tables, reverse mappings, page cache states, memory accounting, and reclamation metadata throughout this lifecycle. Thus, preserving native resource management means that resource allocation, mapping, and reclamation remain part of the host Linux execution path, while accesses to confidential container state are selectively protected from the untrusted host.

Host-resident Linux services. Containers rely on host Linux for system calls, page fault handling, signals, file I/O, and inter-process communication. Processes in the same container use native mechanisms such as shared memory, `futexes`, pipes, and sockets rather than communicating through a guest kernel. Therefore, ordinary services continue to be implemented by host Linux. A service that neither accesses private bytes nor changes protected state should not require entering a guest OS, switching to a separate protected address space, or invoking a higher-privilege security monitor.

B. Mismatch Between Existing Works and Native Containers

Existing confidential-computing systems map container workloads onto different hardware protection units. The resulting systems differ not only in isolation granularity, but also in which software manages process and resource lifecycles and which execution path is taken when a protected workload requests Linux services. Table I summarizes these differences.

Enclave-based approaches. Enclave systems protect an application address space or a designated memory region. Systems such as SCONE, Gramine, Occlum, Keystone, and Penglai provide Linux functionality through enclave runtimes, library OSes, or protected host interfaces [5], [9], [17], [26], [39]. Their hardware protection unit, however, remains an enclave rather than a dynamically changing group of host-managed Linux processes. Multi-process abstractions and resource-sharing semantics must consequently be implemented inside the enclave runtime or coordinated across the enclave boundary. Although these systems can execute containerized applications, their process and resource lifecycles do not directly coincide with those of native host containers.

CVM-based approaches. Confidential VM (CVM) technologies such as AMD SEV, Intel TDX, and Arm CCA protect a guest execution environment from an untrusted host [4], [21], [38]. Kata Containers and Confidential Containers use this protection by placing a container or pod, together with a guest kernel and supporting services, inside a CVM [14], [15], [35]. Ordinary application system calls may be handled entirely by the guest kernel and therefore do not necessarily cause VM exits. Nevertheless, process creation, virtual-memory management, and Linux resource accounting occur inside the guest, while physical-resource allocation and virtualized I/O remain coordinated with the VMM and host. The CVM, rather than the host-visible container process group, is therefore the hardware protection and resource-management boundary.

Arca explores a different TEE-in-Container organization that keeps orchestration logic outside the trusted execution environment and instantiates an independent TEE for each workload [28]. Since Arca supports SGX, TDX, and SEV backends, its concrete protection unit and service path remain determined by the selected enclave or CVM mechanism.

Container-granularity approaches. Recent approaches move the protection boundary closer to containers by protecting container processes or runtimes while retaining selected host services. TZ-Container, BlackBox, ConMonitor, RContainer, and Fasco explore different combinations of trusted execution environments, virtualization extensions, and container-level protection mechanisms [20], [43], [44], [46], [48]. Despite their different implementations, these approaches introduce a separate protection context or mediation layer between containers and the host OS. As a result, frequent container operations may require transitions across nested page tables, protected memory contexts, or trusted execution layers. These additional transitions break the direct execution path between container processes and host Linux, complicating native re-

Table I: Comparison of protection units and Linux service paths in confidential-container approaches.

Approach	Protection Unit	Process and Resource Lifecycle	Linux Service Provider	Common Service Path
Enclave-based [5], [9], [17], [26], [39]	Enclave / address space	Enclave runtime and host OS	Library OS, enclave runtime, and/or host OS	In-enclave handling or enclave–host transition
CVM-based [4], [14], [15], [21], [35], [38]	Confidential VM	Guest OS and VMM/host	Guest kernel	Guest-kernel path; VMM/host involvement for virtualized resources and I/O
TEE-in-Container [28]	Backend-specific enclave or CVM	OCI runtime outside the TEE; workload runtime inside	Backend dependent	Backend-specific enclave or VM path
BlackBox [44]	Protected physical address space	Host Linux under a confidential security monitor	Host Linux through that monitor	EL2 interposition and protected-address-space switching
ConMonitor [46]	Per-container nested-page-table context	Host Linux under ConVisor control	Host Linux and Container Guardian	VMFUNC-assisted nested-page-table switching
RContainer [48]	Container and con-shim protected by guarded page tables	Deprivileged host Linux and trusted mini-OS	Host Linux through mini-OS	Mini-OS mediation with EL3-assisted page-table switching
Fasco [43]	Container Realm	System Realm and Realm Management Monitor	System Realm and Trim OS	Monitor-mediated Realm transition and shared buffers
NACRE	Container identity	Host Linux under identity-based protection	Host Linux through trusted agent	CID-directed S-mode handoff; M-mode only for private data or protected commits

source management and increasing the overhead of frequent container operations.

C. RISC-V Support and Missing Primitives

On a conventional non-virtualized RISC-V system, applications execute in U-mode, Linux executes in S-mode, and platform firmware or a security monitor executes in M-mode. M-mode may delegate selected exceptions and interrupts to S-mode through the `medeleg` and `mideleg` registers. Once a trap is delivered to S-mode, `stvec` determines the S-mode trap-vector base address. The `satp` register selects the root page table, address-space identifier, and virtual-memory translation mode. S-mode software may request services from an M-mode execution environment through the Supervisor Binary Interface (SBI) [36], [37].

These mechanisms separate software by privilege level or virtual address space, but they do not directly represent a protected container identity. Placing all mediation logic in M-mode would cause frequent container events to enter the highest privilege level and would enlarge the highly privileged trusted computing base. Placing the mediator in U-mode would leave its execution and memory mappings under the control of an untrusted S-mode kernel. A native confidential container instead requires trusted mediation at S-mode efficiency while preventing host Linux, which also executes in S-mode, from accessing or modifying the mediator.

Same-level protection-domain isolation. Physical Memory Protection (PMP) allows M-mode to define physical-address permissions for less-privileged software [36]. However, standard PMP checks depend on privilege level and physical

address. They do not distinguish two software components that both execute in S-mode. PMP can therefore isolate M-mode from S-mode, but it cannot continuously distinguish trusted S-mode agent accesses from untrusted S-mode Linux accesses without reconfiguration by higher-privilege software. Native confidential containers require an additional hardware-recognized execution identity that applies different access permissions to same-privilege components.

Container-identity-aware trap routing. Standard RISC-V trap delegation selects a destination privilege level according to the event type, and `stvec` provides one active S-mode trap-vector configuration for a hart. Neither mechanism selects a handler according to the identity of the U-mode process that caused the event. Consequently, traps from an ordinary host process and a confidential-container process follow the same S-mode entry path. Routing all such traps first through M-mode would provide trusted dispatch but would place M-mode on every common event path. Native confidential containers instead require hardware to route a trap according to both its cause and the container identity of its source.

Lightweight same-level handoff. Separating the trusted agent from host Linux with different address spaces would ordinarily require changing `satp`. Although writing `satp` does not itself mandate a complete TLB flush, changing address spaces and reusing translation state may require `SFENCE.VMA` operations and associated TLB synchronization. More importantly, address-space switching would place a new translation context on every mediated service request. Entering M-mode to change protection state would introduce an additional privilege transition. The required mechanism should instead transfer control

between the trusted agent and host Linux while retaining the current address-translation context, and avoid M-mode when the delegated service accesses neither private bytes nor protected state.

III. THREAT MODEL

We consider each confidential container an independent protection domain identified by a `cid`; processes and threads within the same `cid` share a trust boundary. Protected state includes private memory, register contexts, protected page tables, task/address-space bindings, and the security metadata maintained by the trusted agent and security monitor. Data explicitly released to an untrusted service or device leaves this boundary unless an end-to-end protocol protects it.

The adversary controls host Linux, container runtimes, management software, and drivers, and may execute arbitrary S-mode code. It can manipulate scheduling, memory allocation, candidate page tables, traps, identifiers, and host-service results, and it may run malicious containers. Host-provided metadata, mappings, pointers, and resource states are therefore untrusted proposals: they cannot by themselves authorize a protected-state transition. The trusted agent mediates container traps before delegating selected services to Linux, but this mediation does not guarantee the correctness or availability of those services.

We trust the processor and NACRE architectural mechanisms, the OpenSBI security monitor, the trusted agent, the platform boot chain, and the cryptographic primitives used by deployment. Container owners are trusted to provide valid images, keys, and policies, and code admitted to one `cid` is mutually trusting. We assume authenticated boot establishes the monitor and agent before Linux runs, and that an IOMMU or equivalent mechanism prevents host-controlled DMA from reaching protected memory. A device-bound root key and freshness mechanism are required for remote attestation. The QEMU prototype assumes, rather than implements, these platform properties.

NACRE targets confidentiality and integrity of protected container state, isolation among different `cids`, and controlled mediation of container–host interactions. It does not prevent denial of service, starvation, or malicious termination, nor does it isolate mutually distrustful code placed inside one `cid`. Physical attacks, hardware or cryptographic failures, microarchitectural side channels, rollback-resistant persistent storage, traffic analysis, and protection after plaintext is released for untrusted I/O are outside scope. §VI separates this target contract from the properties exercised by the current prototype.

IV. DESIGN

A. Design Overview

Design scope. This section presents the NACRE architecture and its security contract. §V identifies the subset realized by the QEMU prototype.

Design principles. NACRE uses a confidential container, rather than an individual process, as its protection domain.

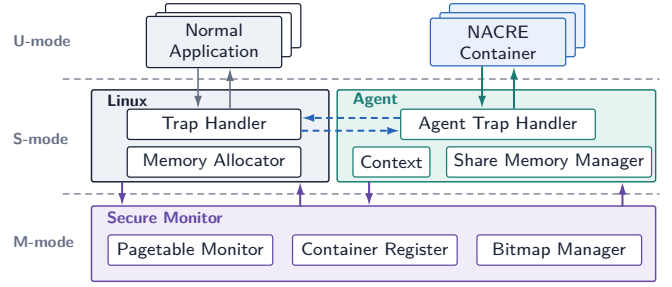


Fig. 1: The NACRE architecture. A confidential container remains a U-mode Linux process, but its traps first enter an isolated S-mode trusted agent. The M-mode monitor maintains security state that neither S-mode component may modify directly.

In the target architecture, a common `cid` identifies the processes, threads, and authorized shared mappings within one container, whereas a cross-container mapping requires explicit authorization. The protection boundary therefore matches the Linux container abstraction without turning every process into an independent confidential execution environment.

NACRE preserves Linux’s resource-management responsibilities without granting Linux security authority. Host Linux continues to create and schedule processes, allocate physical pages, resolve ordinary faults, and maintain VMAs, reverse mappings, and resource accounting. Trusted components instead validate and commit operations that affect confidential context, protected mappings, page ownership, or sharing relationships. This division retains the native container lifecycle while preventing Host Linux from obtaining confidential state merely by modifying pagetables or kernel metadata.

Trusted mediation applies only where a security-sensitive transition requires it. For frequent system calls and exceptions, a trusted agent first saves the confidential context and exposes to Linux only the state needed to perform the delegated host service. The M-mode monitor maintains the security state for protected pagetables, execution bindings, and, in the target multi-container design, page ownership and sharing. Ordinary processes and non-confidential containers retain the direct Linux path. This separation avoids moving general host services into M-mode and leaves the trap path of ordinary workloads unchanged.

Architecture. Figure 1 shows the three-layer NACRE architecture. Ordinary applications and confidential containers both run as U-mode Linux processes, but use different trap entries. Ordinary applications enter Linux directly. A confidential-container trap first enters a trusted agent that, like Host Linux, runs in S-mode. The agent saves registers and other confidential context before Linux gains control, and determines which service state is exposed to the host.

Although the trusted agent and Host Linux both execute in S-mode, they have different authority. Linux schedules tasks, allocates memory, and provides general file and network services. The agent handles confidential-container traps, saves

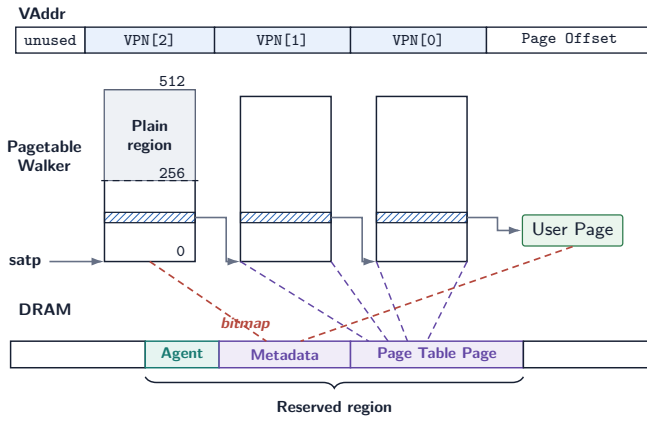


Fig. 2: The NACRE memory-protection layout. Reserved memory contains the trusted agent, monitor metadata, and protected non-root pagetables. Address translation checks each mapping against the execution identity and physical-page role.

and restores confidential context, and prepares the state delegated to Linux. In the target sharing design, it also maintains local policy state for controlled shared objects. The agent cannot modify protected pagetables or monitor metadata. It is part of the TCB, however, and the current access policy permits agent-state execution to read or write private container data when mediating a service. Linux cannot access the agent’s code, data, or saved context.

The M-mode monitor is the commit boundary for security state. It registers container and task identities, maintains page roles, reference counts, and protected pagetables, and validates operations that affect mappings or execution identity. The multi-container design extends this metadata with page ownership and sharing authorization. A delegated host service avoids M-mode when it neither accesses private bytes nor changes protected state; typed private-data operations and protected-state commits enter the monitor. The next subsection describes the mechanisms that support this division of authority.

B. Protection Substrate

The protection substrate combines execution identity with physical-page state. Execution identity distinguishes an ordinary Linux task, the trusted-agent path of a confidential container, and a delegated host-service path. Page state determines whether Linux may access a physical page and which role that page plays in a protected address space. Trusted hardware or the monitor maintains both forms of state. Task flags, VMAs, and candidate PTEs supplied by Linux describe requests but do not authorize them.

Trusted regions and execution identity. As shown in Figure 2, reserved memory contains the trusted agent, monitor metadata,



Fig. 3: Logical execution state carried by NACRE_STATUS.

Table II: Memory-protection metadata.

(a) PMP permissions		(b) Page roles		
Object	Permission	Bits	Role	Meaning
Metadata	M(RW)	00	NORM	Ordinary Linux page
Pagetable pages	M(RW) SU(R)	01	ROOT	Root pagetable page named by satp
		10	PRIV	Confidential user-data page
		11	PEND	Confidential copy in a monitor transaction

and protected non-root pagetables, none of which Host Linux may modify directly. Only M-mode may read and write monitor metadata, so Linux cannot change page roles or private reference counts. S/U-mode translation may read the protected non-root pagetable region, but only M-mode may write it. PMP protects the agent region during boot. After container initialization, the monitor-controlled NACRE_START_AGENT and NACRE_END_AGENT registers delimit its physical range; hardware combines these bounds with the current execution state to admit agent accesses and reject ordinary S-mode accesses from Host Linux.

NACRE_STATUS exposes the registered cid and current control-flow state to the hardware, distinguishing trusted-agent execution from delegated Linux execution. An unregistered task has a zero status and follows the ordinary Linux access and trap rules. A registered task can change this state only when the monitor restores its thread context or through the controlled call and return instructions. In the target architecture, a scheduler hint supplied by Linux therefore cannot open a confidential execution window or grant Linux access to the agent or confidential pages. The prototype boundary for this invariant is described in §IV-C.

Protected address spaces. Linux continues to allocate the root pagetable page named by satp and the user-data pages that back ELF segments and anonymous mappings. When these pages enter the protected lifecycle, the monitor records them as ROOT and PRIV, respectively. A PRIV page is user-accessible only in a registered confidential-container execution state; Host Linux cannot read or write it directly. Lower-level user pagetable pages do not come from Linux. The monitor allocates them from the protected pagetable region, whose PTEs hardware may read during translation but only the monitor may update. PEND is the deny state for a confidential destination while the monitor copies and installs it. Only M-mode can access such a page. A successful copy transaction commits the destination PTE and promotes the page to PRIV; an uncommitted destination must not reenter the ordinary Linux lifecycle. The prototype’s incomplete failure recovery is discussed in §VI.

RISC-V Linux places a process’s user address space in the lower half of the canonical address space and shared kernel mappings in the upper half [18]. NACRE therefore protects only lower-half entries of the root pagetable and leaves Linux in control of upper-half kernel mappings. The monitor must commit all lower-half user PTE updates. Separating ROOT

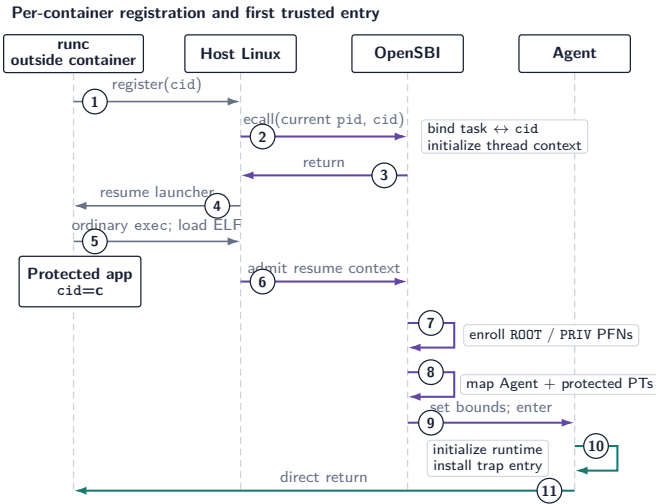


Fig. 4: NACRE per-container runtime initialization and first agent entry after trusted setup. Linux follows the ordinary ELF loading path, while protected execution is admitted only after monitor validation and commit.

from `PRIV` preserves Linux’s control over kernel mappings without allowing it to alter confidential user mappings. The current prototype implements this rule for Sv39. The protection rule itself extends to Sv48 and Sv57 by applying the same checks at their additional pagetable levels; those modes are not implemented or evaluated in the current prototype.

Translation and protected-state commit. On a TLB miss, the current QEMU model selects the translation context from `NACRE_STATUS`. PMP and the agent-region bounds protect intermediate pagetable-walk accesses, while the final physical address is additionally checked against the page-role metadata. A translation enters the TLB only when these checks permit the access. QEMU records a status-derived mode in the translation context, so a later hit cannot be reused across an incompatible mode. Controlled transitions between the trusted agent and Linux therefore do not require a `satp` change or a wholesale TLB flush.

The monitor commits every change to protected page state or protected pagetable permissions. Linux may perform the VMA lookup, classify the fault, allocate a physical page, and prepare a candidate PTE. Before that proposal becomes effective, the monitor checks it against the registered execution context, root pagetable, destination slot, and applicable page-role policy. The prototype mediates generic updates at this boundary; the typed fork-copy and COW-replacement operations additionally combine copy and PTE publication into one monitor transaction.

C. Container Initialization and Runtime Mediation

Container initialization. NACRE separates trusted setup from per-container initialization. The target deployment installs the monitor and agent into the reserved region in

Figure 2 before Linux is treated as untrusted; the QEMU prototype assumes this pretrusted initial state. Initialization is the first container lifecycle transition: Linux-created process state becomes NACRE-protected state, but the process is still created by the ordinary `runc` and Linux `exec` path.

Figure 4 shows this runtime path. Before `exec`, `runc` issues a registration system call (①). Linux forwards the request to the secure monitor. This registration is the initialization gate for a Linux task to become a confidential-container task: the monitor records the Linux task identifier (`pid`) and NACRE container identifier (`cid`) in its registration state and creates the initial per-thread NACRE context (②). The context is keyed by the registered task identity and `cid`, and records the `NACRE_STATUS` value that the monitor will later install when the task is scheduled into the confidential-container execution window. This value encodes the `cid` and the control-flow location used by NACRE to distinguish the `LINUX` state from the `AGENT` state. Tasks without a registered NACRE context use the default zero value. This registration and thread-context state is monitor-maintained state; it is separate from the per-page bitmap used to label physical-page roles. The monitor then returns to Linux (③); Linux resumes `runc` along the same path (④). Registration does not map or execute the agent. This registration call is the only change required to `runc` on the launch path; after it returns, `runc` follows the ordinary OCI lifecycle and invokes the standard Linux `exec` path. Later `fork` and `clone` operations that create additional confidential-container tasks must similarly create registered NACRE thread contexts before those tasks can enter the confidential-container execution window. The QEMU prototype accepts the `pid` and `cid` supplied along this cooperative launch path. It therefore validates lifecycle plumbing, not resistance to forged or stale membership. A deployment must bind accepted identities to monitor-controlled freshness and derive child membership from an authorized parent transition; §VI treats this as an open link in the current security argument.

`runc` later executes the container program and transfers control to the post-`exec` application image. Linux handles the ordinary `exec` path and builds the user address space from a prepared ELF image (⑤). The current QEMU prototype treats this image as a pretrusted test input. In the target deployment, an authenticated image-dispatch path must bind the protected contents and public loading metadata to a manifest before the monitor admits protected execution; that image-verification protocol is not implemented in the prototype.

Control then enters the monitor with the application’s user-mode resume context (⑥). Before the agent is mapped, the monitor uses its page-monitor logic to enroll the pages allocated during ELF loading into protected metadata: `ROOT` for the root pagetable page and `PRIV` for Linux-allocated user data pages (⑦). This enrollment also initializes the private reference count for committed user-data mappings and rejects physical frame numbers (PFNs) that already carry an incompatible role. The security monitor then updates the target process pagetable and maps the reserved agent region into a user-space slot in the low-half part of the NACRE process

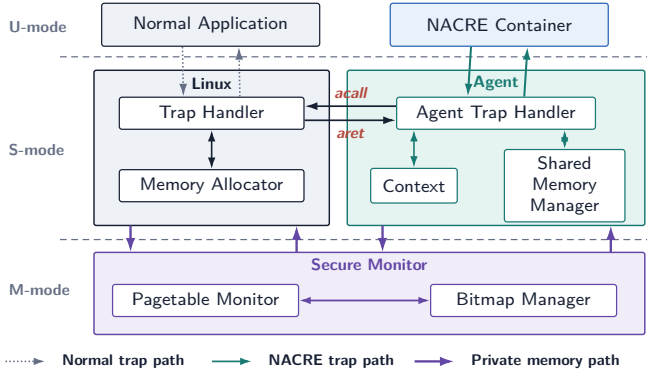


Fig. 5: NACRE post-initialization runtime paths. Normal applications use the normal trap path, confidential-container traps use the NACRE trap path through the agent, and private-memory transitions enter the monitor when protected pagetable state, bitmap/refcount metadata, or confidential data-page copy/retirement must be committed.

address space (⑧). This slot is chosen to avoid the ranges normally occupied by the ELF image, heap, stack, and Linux-managed anonymous mappings, so applications usually do not need binary or source changes. The agent mapping is not part of the ELF image; it is installed only at this monitor-mediated mapping step. Although the agent is mapped into a user-address-space range, the pagetable pages carrying this mapping are not allocated or modified by Linux. The monitor allocates them from the protected pagetable page area in Figure 2, following the same monitor-only update rule used for protected user mappings. Before entering the mapped agent, the monitor switches protection from the boot-time PMP guard to two monitor-controlled registers, `NACRE_START_AGENT` and `NACRE_END_AGENT`, that Linux cannot modify. These registers bound the agent physical region (⑨). Once this base-and-bound guard [22] is enabled, M-mode and S-mode execution in the `AGENT` state may access the region, whereas protected U-mode and Linux S-mode accesses are rejected. The state transition, not S-mode privilege alone, therefore distinguishes the agent from Linux.

The agent then initializes its own address-space memory and local runtime state so it can later handle container requests (⑩). As part of this setup, the agent records the virtual entries of the original Linux trap handlers used for delegated services, and writes the virtual entry of its own trap handler into `NACRE_TWIN_ENTRY`. The latter becomes the first-hop target for later delegated traps from confidential-container execution. When this completes, the agent jumps directly back to the user application using the application resume context saved at (⑥), without re-entering the security monitor (⑪).

Figure 5 summarizes the runtime control paths after initialization. We first focus on the two trap paths: the normal trap path keeps ordinary applications on the Linux trap handler, while the NACRE trap path routes confidential-container traps through the agent before Linux executes the delegated service.

Scheduling window and thread binding. The distinction between the first two paths in Figure 5 is made by the execution window restored for the scheduled task. After initialization, the monitor uses the registered NACRE thread contexts to decide when a scheduled Linux task may enter the confidential-container execution window. Linux still saves and restores ordinary process context through its normal scheduler. The scheduling state specific to NACRE, including the encoded `cid` and control-flow location in `NACRE_STATUS`, is recovered from the monitor-registered thread context rather than trusted from Linux at context-switch time.

In the QEMU prototype, Linux marks candidate confidential-container tasks in `task_struct` and enters the monitor when a switch may involve one of them. The monitor then saves the outgoing NACRE state or restores the registered incoming state, including `NACRE_STATUS`. This cooperative path keeps ordinary context switches unchanged, but the Linux marker is not a security authority: a malicious scheduler can omit the hook. The target invariant is that a protected user return is accepted only for a fresh, monitor-bound task, `cid`, and address-space context. The current prototype does not yet provide the non-bypassable scheduling hook and freshness mechanism needed to enforce that invariant against malicious Linux.

Trap delegation and fast-path separation. The normal and NACRE trap paths in Figure 5 differ only when a registered confidential-container context is active. Normal tasks keep the unmodified Linux path: hardware vectors their system calls and exceptions through `stvec` to the Linux trap handler. A registered NACRE task is distinguished by a nonzero `NACRE_STATUS`. When such a task traps inside the confidential-container execution window, the first hop goes to the agent entry stored in `NACRE_TWIN_ENTRY` rather than directly to Linux.

The agent saves the protected register context and builds a delegated context in Linux-readable memory. It then uses `scause` to select one of the Linux trap-handler entries recorded during initialization and writes that handler address into the `NACRE_TRAMPOLINE` register. Executing `acall` switches `NACRE_STATUS` to the `LINUX` state and transfers control through that trampoline to the selected Linux handler. As part of this transfer, the trampoline is changed to hold the agent return entry, so the later return has a single controlled target. Linux then runs its ordinary handler over the delegated context, without direct access to the protected container context.

When Linux finishes the service, its common return wrapper executes `aret`. For a valid pending return in the `LINUX` state, the instruction clears the pending state, transfers control through the trampoline, and restores the `AGENT` state. The agent restores the Linux-produced trap frame after clearing `SIE` and `SPP` and reseeding `sscratch`; it does not validate the semantics of arbitrary host-service results. Because the agent and Linux use the same process pagetable, `acall` and `aret` do not change `satp`, switch pagetables, or flush TLB

entries. A delegated service also avoids M-mode when it neither accesses private bytes nor changes protected mappings or metadata. Private-buffer access instead uses the typed monitor operations described in §V.

The result is a separation between the two trap paths: ordinary tasks trap directly to Linux, while confidential-container traps return through the agent. Monitor entries for private-byte access and protected-state changes are outside the `acall/aret` fast path and use the private-memory path described next.

Core private-memory substrate. The private-memory path in [Figure 5](#) first establishes the single-container substrate: monitor entries commit protected page-role or mapping changes for one confidential container. The boundary is the protected commit point. Linux proposes VM policy results through the ordinary kernel path: it classifies faults, looks up VMAs, allocates backing pages, and maintains `struct page`, reverse mapping, RSS, and LRU state. When the kernel path reaches a result that would update a confidential mapping, copy confidential contents, or retire protected state, control enters OpenSBI through an SBI `ecall` carrying the operation type, target PTE location, and intended mapping. OpenSBI uses the registered task and root context as validation inputs before it performs the protected pagetable update or data-page operation. VMA flags and Linux PTE contents route the request, but they are not security authority.

OpenSBI records per-page protected state in the page metadata shown in [Figure 2](#). The bitmap values in [Table IIb](#) distinguish NORM pages, PRIV confidential data pages, and ROOT root pagetable pages. Private reference counts are stored alongside this per-page state and are writable only by OpenSBI. This counter is separate from Linux’s `mapcount`: it records the monitor-approved protected mappings that currently hold each confidential PFN, and it is used to decide when a PRIV PFN can be retired. The bitmap is therefore only a per-page role encoding for the protected-memory substrate; it does not encode a confidential container context, a thread context, or a complete ownership policy. Those states are maintained separately by the monitor. More expressive shared-object or multi-container policies can be layered on top of this substrate without changing the basic rule that OpenSBI is the protected-state committer.

For a protected page fault, Linux performs VMA and fault resolution and obtains the physical page together with its ordinary `struct page` and accounting state. The PFN remains NORM at this point. Once the prepared leaf PTE targets a protected user mapping, OpenSBI commits the transition. In the target protocol, it verifies that the PTE slot belongs to the registered root pagetable page or to a protected non-root pagetable page, checks that the target PFN can enter the protected lifecycle for this registered address space, records the PFN as PRIV, initializes or increments its private reference count, and writes the PTE. The root pagetable page and ordinary user data pages are still allocated by Linux; OpenSBI protects them by assigning bitmap roles and mediating pro-

ected updates, rather than by replacing Linux allocation. If the walk needs a new non-root pagetable page, OpenSBI allocates it from the protected pagetable page area and initializes the corresponding pagetable metadata before committing the leaf PTE. The same path handles later demand faults after initialization and after `exec` rebuilds an address space.

Container lifecycle across processes. `fork`, `exec`, COW, and `exit` extend the substrate from a single protected address space to the normal Linux container lifecycle. Linux retains its native `mm` duplication, pagetable traversal, VMA/rmap/RSS accounting, and fault classification. The trusted agent brackets a distinct-`mm fork` with a monitor-owned authorization window, and the monitor admits at most one fresh child root for that operation. This relation prevents a Linux-supplied root or process identifier from authorizing an unrelated protected mapping.

Inherited private pages are copied eagerly rather than shared between the parent and child. For each private parent leaf, Linux prepares a fresh child destination and invokes `fork_copy_install`. The monitor validates the parent root, selected child root, virtual address, source leaf, destination PFN, and requested PTE shape. It changes the fresh destination from NORM to PEND before copying the confidential bytes. Successful commit promotes the destination to PRIV and publishes the child PTE atomically; no child PTE is visible before that commit. The child therefore begins with private copies while Linux still performs the surrounding native `fork` bookkeeping.

Linux also retains its ordinary writable-fault path for a protected leaf that requires COW replacement. It allocates and accounts for the destination page, then invokes the monitor’s `cow_replace` transaction. The monitor validates the source and destination, changes the destination from NORM to PEND, copies the confidential bytes, and atomically replaces the protected PTE while promoting the destination to PRIV and retiring the old protected reference. Both operations collapse what would otherwise be a separable copy and PTE update into one monitor commit; they do not rely on a Linux-controlled copy reservation.

The target recycle and retirement protocol closes the lifecycle. During `munmap`, COW replacement, `exec` teardown, or process exit, Linux may still walk VMAs and pagetables and reach the kernel sites that would normally clear PTEs. OpenSBI validates each retired protected PTE, clears or replaces it, and decrements the private reference count of the mapped PFN. When the count reaches zero, OpenSBI first ensures that no protected PTE still holds the PFN, clears the confidential contents, and only then returns the bitmap entry to NORM so the PFN can reenter the ordinary Linux lifecycle. During address-space destruction, batching is limited to protected non-root pagetable pages whose protected leaf entries have already been retired. These pages return to the protected pagetable page area rather than the Linux buddy allocator. The root pagetable page is retired last by removing its ROOT bitmap entry.

Same-container controlled sharing. Explicit shared-memory IPC is a separate layer above the private-memory and multi-process lifecycle substrate. Eager private-page copying during `fork` deliberately does not provide a shared-memory object model. In the same-`cid` design, the agent acts as the policy and control plane for approved shared objects. The agent may mediate object data, but it cannot commit protected PTE or monitor-metadata changes. For each approved attachment, OpenSBI records the object, `cid`, permissions, and live mapping before installing the protected PTE. Detach revokes the mapping and stale translations before the backing page can be reused. Linux therefore cannot turn a private PFN into a shared object by writing PTEs alone.

Cross-container ownership and isolation. Cross-container isolation adds owner and generation metadata to protected non-root pagetable pages and private PFNs. Every protected PTE update is bound to the registered task, `cid`, root pagetable page, virtual address, target PTE slot, and object grant, if any. A request to map, retire, or copy another container’s protected PFN is rejected unless a live monitor-visible grant names the recipient and permissions. Revocation removes authorized mappings and stale translations before the object generation can be reused.

Membership transitions. Container membership changes decide which tasks and address spaces may invoke the private-memory transitions above. A `clone` that shares the address space adds a task to the same `cid`, so the new thread remains inside the existing protection domain only after the monitor records the new NACRE thread context. A `fork` creates a new address space, so its root pagetable page and inherited protected mappings must be derived through monitor-checked metadata transitions. The child is not allowed to enter the confidential-container execution window until its task identity, `cid`, and `NACRE_STATUS` value have also been registered as a NACRE thread context. Futex synchronization then operates on already-approved shared pages rather than defining a new protection boundary.

D. Target Measurement and Attestation

NACRE defines a measured container object over the monitor and agent configuration, initial application image, accepted public mappings, and container policy. Attestation binds this object to a fresh `cid` and root-pagetable generation; later reports describe only state changed through the protected transitions above. This lets a verifier reason about the Linux container as one protection domain rather than attest a collection of unrelated processes. Report generation and verifier integration are outside the current prototype.

V. IMPLEMENTATION

The design section defines the protection rules and runtime paths. This section describes where those rules are realized in the prototype. We focus on implementation boundaries rather than restating the design: which components were changed, which state is kept by Linux versus OpenSBI, and which Linux events are converted into monitor-mediated commit points.

A. Prototype Organization

Our prototype combines modified QEMU 9.2.0, OpenSBI derived from v1.6, Linux, a trusted agent, and a minimally changed `runc`. The RISC-V guest uses an Ubuntu 22.04 root filesystem. QEMU serves as the architectural and functional model; §VII reports the corresponding experimental scope.

The prototype is organized to keep ordinary Linux container paths intact. `runc` is modified only to issue one registration system call before `exec`; the remaining OCI launch path is unchanged. Linux changes are localized: ordinary `exec`, scheduling, VM policy, and kernel metadata remain on their standard paths, while NACRE adds branch checks and `ecall` exits at points that access private bytes or commit protected state. OpenSBI adds SBI handlers that manage protected metadata, protected pagetable updates, and registered context state. QEMU models the NACRE execution-state, twin-entry, trampoline, and agent-bound registers; `acall/aret`; agent-region checks; and final-address page-role checks. The corresponding QEMU fields are `nacc_state`, `twin_entry`, `trampoline`, `nacc_sagent`, and `nacc_eagent`. The agent is a trusted runtime for trap delegation and service preparation. Agent-state execution may access private container data, but it cannot commit protected pagetable or monitor-metadata changes; those operations remain within OpenSBI.

The rest of this section follows this component boundary. We first describe boot-time agent installation and per-container registration, then describe the monitor metadata, commit interfaces, private-memory transitions, and modeled architectural support used by the prototype.

B. Boot and Launch Integration

NACRE builds a normal Linux boot artifact and appends two objects to it: the agent payload and a descriptor page. The descriptor records the location of the appended agent image and the reserved physical region where the image must be installed. The QEMU prototype assumes that this artifact and the initial container image are pretrusted inputs; it does not implement the target authenticated-boot or image-manifest protocol.

OpenSBI performs the installation before transferring control to Linux. It parses the descriptor, copies the agent payload into the reserved agent region shown in Figure 2, clears both the appended agent copy and the descriptor from the original boot artifact, and installs a temporary PMP rule that makes the loaded agent region accessible only to M-mode. Linux can then continue its ordinary boot path, but it cannot read or overwrite the installed agent bytes.

Per-container launch therefore adds only one hook to the ordinary container path. Before `exec`, `runc` issues the registration system call described in §IV-C, carrying the NACRE container identifier (`cid`). Linux associates the request with the current task and forwards the resulting `pid/cid` tuple to OpenSBI. After the call returns, `runc` resumes the ordinary OCI lifecycle, and Linux still performs the subsequent `exec` and ELF loading work.

Table III: Representative NACRE SBI operations (paper-facing names).

Call	Role in the protected lifecycle
NACRE_REGISTER	Records the initial <code>pid/cid</code> binding before <code>exec</code> .
NACRE_INVOKE	Enters the agent path for the registered task and establishes the runtime execution context.
NACRE_REEXEC	Reattaches after <code>exec</code> , when Linux has built a fresh <code>mm</code> and root pagetable page.
NACRE_INVOKE_CHILD	Invokes an already initialized child process through a shortened <code>invoke</code> path.
NACRE_ATTACH_FORKED_CHILD	Attaches a freshly forked child before its first protected user-mode return, skipping the initial bootstrap.
NACRE_VERIFY	Saves or restores monitor-owned thread context at scheduler boundaries.
NACRE_TAG_ROOT	Enrolls the current address-space root pagetable page for protected updates.
NACRE_REQ_PTP	Allocates a protected non-root pagetable page from monitor-managed memory.
NACRE_SET_PTES	Commits Linux-prepared PTE values after OpenSBI validates the target slots.
FORK_BEGIN/FORK_END	Opens and closes the monitor-owned authorization window around one Agent-observed distinct-mm fork.
FORK_SELECT_CHILD_ROOT	Selects at most one fresh child root for the active parent fork window.
FORK_COPY_INSTALL	Copies one exact parent private leaf into a fresh child page and publishes the child PTE as one monitor transaction.
COW_REPLACE	Copies and replaces an exact private leaf within one attached address space as one monitor transaction.
NACRE_RETIRE_ROOT	Retires state for the root pagetable page when the protected address space is replaced or destroyed.
NACRE_RECLAIM_PTP	Batch-reclaims protected non-root pagetable pages after leaf mappings are gone.
NACRE_UNREGISTER	Removes monitor-side task registration state during process exit.
NACRE_AGENT_MMAP	Maps the agent bootstrap region through OpenSBI during the first agent entry.
NACRE_AGENT_READY	Marks the first full agent initialization complete, enabling later lightweight attaches.

C. NACRE SBI Commit Interfaces

The implementation exposes the design’s protected-state commit points through NACRE-specific SBI ecalls. We use the paper-facing operation names in Table III; the concrete Linux and OpenSBI trees group these operations under their respective NaCC SBI extension identifiers. These interfaces are not a general metadata management API: they are the places where Linux, after following an ordinary container or memory-management path, must enter OpenSBI before a protected binding, pagetable update, or teardown action becomes effective.

Table III lists the representative monitor entries used by the prototype. These calls appear at the points where the native Linux lifecycle attaches a registered task, installs or updates a protected PTE, or retires protected address-space state.

D. Single-Container Lifecycle Integration

Registration and attachment. NACRE splits process identity registration from protected address-space attachment. The registration call made by `runc` creates or refreshes the monitor-side `pid/cid` record and per-task execution context, but it does not yet attach a specific `mm`. After `exec` succeeds, Linux has constructed the new address space and

root pagetable page, and the task enters OpenSBI through the `invoke`, `reexec`, or `child attach` path. The first protected task performs the full agent bootstrap and marks the agent runtime ready with the agent-side calls in Table III. Later `exec` and `child` paths reuse this installed runtime state: `NACRE_REEXEC` refreshes the root and trap-return context for the same registered task, while `NACRE_INVOKE_CHILD` and `NACRE_ATTACH_FORKED_CHILD` are shortened variants for child processes that avoid repeating the first-process bootstrap.

Protected PTE installation. Linux still owns the software memory-management state: VMAs, `struct page`, RSS, reverse mappings, LRU state, and `memcg` accounting remain on the native paths. The implementation changes the physical source and write authority for protected pagetable pages. The root pagetable page is a Linux-allocated page that OpenSBI tags as `ROOT`; non-root protected pagetable pages are allocated from the monitor-managed PTP pool while Linux keeps the ordinary page-table descriptors needed by its VM code. Instead of adding separate security logic to every fault, fork, `mprotect`, and `unmap` branch, NACRE redirects common RISC-V PTE primitives—install, write-protect, exchange, and accessed-bit updates—when the target slot belongs to protected pagetable state. Linux supplies the candidate PTE, target slot, virtual address, and root context; OpenSBI validates the destination and PFN role, updates the bitmap/reference-count metadata, and writes the protected PTE.

Fork and COW. `fork` continues to use Linux’s native `mm` duplication, page-table traversal, VMA/`rmap`/`RSS` accounting, and surrounding fault logic. At the trusted trap entry, the agent recognizes a legacy clone with `CLONE_VM` clear and opens a monitor-owned fork window. Linux then submits the root allocated by its normal `pgd_alloc()` path. OpenSBI admits at most one fresh child root for the window and records that exact parent-child relation; Linux-supplied task or container identifiers remain consistency inputs rather than authority. The child cannot enter its protected execution window until its task context and selected root are attached.

The implemented fork path does not publish the parent’s private PFN into the child. When native `copy_page_range()` reaches an inherited protected leaf, Linux prepares a fresh destination page and invokes `FORK_COPY_INSTALL`. Under the monitor lock, OpenSBI validates the active fork window, selected child root, parent source leaf, child destination slot, virtual address, PFN roles, and PTE shape. It changes the fresh destination from `NORM` to `PEND`, copies the page in `M-mode`, and then commits the child PTE together with the `PEND-to-PRIV` promotion as one transaction. Thus the parent and child receive distinct private PFNs even though Linux retains its native fork bookkeeping. The agent closes the fork window when the delegated clone returns; closing the window neither retires nor detaches an admitted child.

For a later private COW fault within an attached address space, Linux still allocates and charges the destination page and performs its ordinary race check under the PTE lock. It

then invokes `COW_REPLACE` with the exact slot, expected old PTE, and candidate destination PTE. OpenSBI derives the source PFN from the observed protected leaf and validates the fresh destination. It then changes the destination from `NORM` to `PEND`, copies the bytes, and commits the replacement leaf together with the `PEND-to-PRIV` promotion and private-reference updates. `PEND` is a transient deny state inside these operations; the implementation maintains no persistent Linux-controlled copy reservation.

Teardown and recycling. `munmap`, `COW` replacement, `exec` teardown, and process exit reuse the same mediated PTE-clear and exchange sites. OpenSBI retires the private reference associated with the old protected leaf before Linux completes its normal `RSS`, `rmap`, `TLB`, and free-page bookkeeping. Protected non-root pagetable pages are not returned to the Linux buddy allocator: Linux queues their PFNs in a reclaim list, and `NACRE_RECLAIM_PTP` returns them in batches to the monitor-managed `PTP` pool after their leaf mappings have been retired. The root pagetable page follows a separate lifetime because it was allocated by Linux and carries the root role; it is retired after the protected leaves and non-root `PTP` pages are gone. Finally, `NACRE_UNREGISTER` removes the monitor-side task record and saved execution context, rather than serving as the address-space cleanup mechanism.

E. Linux Memory Integration

Because the implementation keeps Linux’s ordinary memory accounting and PTE update machinery in place, it must also handle Linux features that inject or access user mappings outside ordinary anonymous faults. NACRE handles two such compatibility points in the prototype: executable `vDSO` code and `S-mode` access to private user bytes.

vDSO integrity. Linux still creates the `vDSO` ABI mapping during `exec`. During NACRE initialization, OpenSBI validates the `vDSO` mapping shape, copies the accepted image into a monitor-managed protected memory region, and maps that copy at the expected user `vDSO` address. The protected process can execute the usual `vDSO` stubs, while Linux cannot modify or replace the backing page after adoption. The associated `VVAR` data page follows a distinct public-mapping rule: OpenSBI binds its root, virtual address, PFN, and read-only, non-executable user permissions, while Linux retains the supervisor mapping used to update the time data. Thus `vDSO/VVAR` compatibility does not require reclassifying arbitrary user pages as public.

S-mode user-memory access. Preserving native Linux VM behavior would become difficult to audit if NACRE added an independent case to every VM and `uaccess` branch. NACRE therefore introduces two Linux-side classification markers to keep the modification surface narrow. `VM_NACC_APP` is a VMA flag for candidate private-anonymous application mappings in an active NACRE address space. `_PAGE_NACC` is a software PTE bit that carries private-data intent in concrete leaf PTEs and across later PTE updates. Together they route fault, `COW`, `uaccess`, and teardown paths toward the protected

OpenSBI interfaces. After a root is attached, OpenSBI defaults ordinary resident user leaves to private independently of their `U/W/X` shape, while explicitly recognized public/system, special, device, and `PROT_NONE` mappings remain outside that class. Linux markers describe candidates and exceptions; they do not authorize protected ownership.

`S-mode` access to user memory appears in two forms. Explicit `uaccess` uses user virtual addresses and normally reaches shared `RISC-V` `uaccess` primitives that bracket `S-mode` access by setting and clearing `SUM`, the `RISC-V` status bit that permits `S-mode` instructions to access user pages. For a candidate private range selected by the `VMA/PT` markers, those primitives enter private-data SBI operations instead of opening a raw `SUM`-enabled access to the page. OpenSBI resolves the current user address, checks the private leaf and requested direction, and performs the read, write, copy, or clear in `M-mode`. Non-private ranges continue on the normal Linux helper path. If a private destination is valid but write-protected for `COW`, OpenSBI reports a locked result; Linux clears `SUM`, enters the ordinary writable-fault/`COW` path, and retries after OpenSBI commits the replacement leaf through the same `PEND-to-PRIV` path used by private `COW`.

Raw `S-mode` dereference is the case where Linux reaches the same PFN through a supervisor mapping, such as the direct map, a `kmap`-like alias, or a low-level load/store fallback path. `SUM` is not the relevant permission gate for these aliases, but the number of legitimate native Linux entry points that need private user bytes is limited. NACRE handles them as explicit OpenSBI operations: Linux enters OpenSBI with the intended access and range, and OpenSBI checks the request against monitor-maintained protected context before performing the copy or update in `M-mode`. This differs from designs that route such interactions through a shared or bounce buffer outside the protected address space. NACRE keeps the native Linux call path, but Linux does not directly dereference or modify `PRIV` PFNs; the data movement is performed only after OpenSBI validates the protected leaf and the requested operation. All other accesses to a `PRIV` PFN are denied by default.

F. Trap and Architectural Support

The trap implementation follows the fast-path separation in [Figure 5](#). Normal tasks continue to use the ordinary Linux trap path. A registered NACRE task traps first into the agent through the entry recorded in `NACRE_TWIN_ENTRY`. The agent saves the protected context, prepares a Linux-readable trap frame, and transfers control to the selected Linux handler through `acall`. Linux returns through a common `arev` wrapper. The agent restores the resulting frame after sanitizing the return `sstatus` fields described in [§IV-C](#); host service semantics remain untrusted. The `acall/arev` pair does not enter `M-mode`, but a delegated service that accesses private bytes invokes the typed OpenSBI operations above.

The QEMU architectural model implements the state needed by this path. `NACRE_STATUS` records the registered container and whether the current confidential-container control flow is executing in Linux or in the agent. The translation path

treats this state as part of the translation context, so entries filled for one NACRE execution state are not reused as valid hits after switching to another state. The model also enforces the agent-region base-and-bound registers. During a pagetable walk, PMP and the agent bounds protect intermediate PTE accesses; the page-role policy is checked for the final physical address before QEMU installs the translation. When OpenSBI changes a page role, QEMU invalidates cached translations for that PFN range across its virtual CPUs before later accesses proceed under the new role.

G. Prototype Scope

The QEMU prototype implements the core substrate rather than the complete target NACRE system. Its implemented paths include monitor-mediated protected pagetable updates, bitmap/reference-count metadata, private anonymous-memory transitions, trap delegation through the agent, and the principal `fork/COW/exit` paths exercised by our functional workloads. In particular, the current `fork` path uses one selected child root and the monitor-owned `FORK_COPY_INSTALL` transaction; it does not depend on shared private PFNs or a persistent copy-reservation protocol.

The prototype does not implement owner-tagged cross-container isolation, a monitor-visible shared-object model, stale-identity protection, or attestation. Its generic PTE interfaces also lack complete exact-slot and purpose authorization, and teardown does not yet provide a malicious-host-safe `revoke-flush-clear-before-NORM` transaction or complete failure rollback. Agent entry is validated only on the cooperative prototype path and has not been established for concurrent multicore execution. These limitations bound the security claims in §VI without changing the prototype’s positive-path behavior.

VI. SECURITY ANALYSIS

We analyze NACRE against the adversary in §III. The target design relies on five invariants:

- **I1: Monitor-only protected commits.** Only the monitor can commit lower-half entries in a `ROOT` page, protected non-root page tables, or page-role and protected-reference transitions. Linux retains authority over upper-half root entries.
- **I2: Linux proposes, but does not authorize.** Linux may run VM policy, allocate pages, and construct candidate PTEs, but the monitor treats these values as untrusted inputs.
- **I3: Bound execution.** Protected user execution requires a fresh monitor binding among a task, `cid`, address-space root, and saved execution context.
- **I4: Domain-bound memory.** A protected page or page-table page may be mapped only by its owning `cid`, unless a live monitor-visible grant authorizes the mapping.
- **I5: Authenticated initialization.** Protected execution begins only after the monitor accepts the initial image, address space, agent, and public-mapping policy.

A. Memory Isolation

An adversarial Linux can try to read a private page through its user mapping, through a direct-map or temporary kernel alias, or by installing a second PTE to the same physical frame. NACRE places the final data-plane check below these virtual-address choices. The architectural model checks the current execution state and final physical-page role: ordinary U-mode and Linux S-mode cannot access a `PRIV` page, whereas protected application and trusted agent execution can. `PEND` pages are accessible only in M-mode. Changing an alias therefore does not bypass the page-role check.

Linux also cannot directly rewrite the relevant translation structures. Protected non-root page tables are S/U-readable for translation but M-mode writable, and only the monitor commits the lower-half entries of a `ROOT` page. Linux retains its normal upper-half kernel mappings. In the target multi-container design, the monitor further binds every protected page, page-table page, root, virtual address, and destination PTE slot to an owner and generation. A wrong-owner mapping or role transition is rejected before either the PTE or page metadata changes. These checks establish I1, I2, and I4 when the owner and exact-slot metadata are complete.

B. Lifecycle Transitions

Protection must hold while a page changes roles. On a private anonymous fault, Linux may allocate and clear a `NORM` page because it contains no application secret. Before the application can access it, the monitor records the private role and reference and publishes the protected leaf. For an attached address space, ordinary resident user leaves default to private unless the monitor recognizes an explicit public, system, special, device, or `PROT_NONE` mapping; Linux’s VMA and PTE markers route the request but do not authorize the exception.

`Fork` and `COW` require a stronger copy protocol because the source already contains confidential bytes. The monitor validates the source mapping and a fresh destination, changes the destination from `NORM` to `PEND`, and performs the copy in M-mode. A successful transaction atomically publishes the new PTE and promotes the destination to `PRIV`. `Fork` uses a fresh child PFN, so parent and child never share the same private PFN; `COW` replaces one exact protected leaf in its existing address space. Neither path exposes a `PEND` page to Linux or relies on a Linux-controlled reservation.

The reverse target transition first removes protected mappings and stale translations, then verifies that the last protected reference is gone, clears the page in trusted mode, and only then publishes it as `NORM`. A failed copy or retire operation must publish neither a partial mapping nor a reusable page containing confidential data. Together these orderings extend I1 across fault, `fork`, `COW`, `unmap`, `exec`, and `exit`.

C. Execution and Trap Mediation

Linux controls scheduling, but task metadata alone must not authorize protected execution. The target design admits a user return only when the monitor-restored task, `cid`, root,

and saved context satisfy I3. A trap from that execution first reaches the agent. The agent saves the private frame, constructs the frame delegated to Linux, and uses `acall` to enter the selected Linux service. A valid pending `aret` returns control to the agent rather than directly to protected U-mode.

The current agent does not establish semantic correctness of arbitrary Linux results. It restores the Linux-produced frame after clearing `SIE` and `SPP` and reseeding `sscratch`; applications must still treat host-service outputs as untrusted. Linux can also return errors, delay the service, or refuse it entirely. These behaviors do not grant direct access to the saved private context, but they remain sources of Iago attacks and denial of service.

D. Mediated Data Access and Sharing

Some Linux services legitimately consume or produce private user bytes. Rather than grant Linux a raw mapping, NACRE invokes typed monitor operations. The monitor resolves the user virtual address, checks the protected leaf and access direction, and performs the copy or clear in M-mode. This mechanism preserves the native Linux call path while making private-byte movement explicit. Its security nevertheless depends on binding each operation to an authorized purpose and range; a malicious kernel must not be able to synthesize an otherwise well-formed copy request for arbitrary data.

Not every user mapping is private. The design admits monitor-recognized public and system mappings, such as immutable vDSO code and a constrained read-only VVAR page, while defaulting ordinary application leaves to private. Writable sharing requires a separate protected object. Same-container attachments name one object and `cid`; cross-container attachments additionally require a grant naming the recipient and permissions. Revocation removes every authorized mapping and stale translation before the object generation can be reused. This object rule prevents Linux from reclassifying an arbitrary private PFN as shared.

E. Prototype Security Boundary

The QEMU prototype provides positive-path evidence for a narrower substrate. Its final-address policy rejects raw Linux S-mode loads, stores, atomic memory operations (AMOs), and cache-block operations (CBOs) to `PRIV` pages and admits `PEND` access only from M-mode. OpenSBI writes the protected page tables and page-role metadata exercised by the functional workloads, while `FORK_COPY_INSTALL` and `COW_REPLACE` combine copy and PTE publication into one monitor transaction. The agent-first trap path and pending `aret` edge are also implemented. §VII evaluates these covered paths.

This evidence does not establish the full malicious-host contract. The current metadata lacks complete owner/generation and exact-slot authorization; the Linux scheduler hook and `pid/cid` registration path are cooperative; typed data operations are not fully purpose-bound; and generic failure rollback

and trusted clear-before-NORM recycling are incomplete. Protected shared objects, cross-container grants, multicore-safe agent entry, authenticated initialization, DMA enforcement, and attestation are also outside the prototype. Thus the current results support the data-plane isolation and positive lifecycle paths above, but not yet a complete proof of confidentiality and integrity against malicious Linux.

VII. EVALUATION

Our evaluation asks four questions: (1) which Linux lifecycle paths does the prototype execute, (2) what cost appears on common system-call and communication paths, (3) can the prototype run service workloads, and (4) which security and scalability claims remain open? We distinguish repeated measurements from functional and diagnostic tests, and report only evidence collected with the QEMU prototype.

A. Experimental Scope and Methodology

The prototype runs in modified QEMU 9.2.0 with a RISC-V Ubuntu 22.04 guest. The recorded experiments compare the NACRE runtime with a native `runc-origin` baseline. For `lmbench`, both arms use one Docker daemon and select either the unmodified `runc-origin` or patched `runc` with `--runtime`. The preserved summaries do not establish the same daemon control for the other workloads.

We use three evidence classes. Repeated measurements report three executions per arm. Functional tests establish only that the checked workload and output complete. Diagnostic tests expose a failure boundary but do not support a performance claim. Because all experiments run in QEMU, the results characterize the current software prototype rather than another deployment environment.

B. RQ1: Which Linux Paths Does the Prototype Execute?

An eight-path functional regression exercises private-buffer reads and writes, anonymous memory, `fork/COW`, file and pipe I/O, `exec`, teardown, and a shared-memory compatibility path. All eight workloads returned `status=ok` and produced their expected outputs. A joint scan of their QEMU and guest logs found no raw-access denial, typed-copy denial, access fault, kernel Oops, or panic. This regression belongs to a revision before the final fork transaction, so it establishes only the covered paths on that revision.

The implemented fork transaction replaces the earlier multi-step path with a monitor-owned `fork_copy_install` transaction. Its dedicated protected-fork test produced the expected output, exited with code zero, and showed no NACRE rejection or guest crash. The two regressions therefore provide complementary evidence: the broad regression covers path breadth on the prior revision, whereas the dedicated test covers the implemented fork path. A complete broad regression on the same revision remains missing. Existing Linux Test Project (LTP) records are dominated by container capability and `seccomp` policy, and NACRE coverage is incomplete; we consequently make no Linux-ABI completeness claim from them.

Table IV: Same-daemon lmbench measurements on QEMU. Values are three-run means; latency is lower-is-better and bandwidth is higher-is-better.

Case	Origin	NACRE	Ratio
null syscall (μ s)	4.969	4.894	0.985
read syscall (μ s)	8.523	8.389	0.984
write syscall (μ s)	5.892	5.924	1.005
pipe latency (μ s)	730.357	705.012	0.965
pipe bandwidth (MB/s)	47.803	47.983	1.004

C. RQ2: What Cost Appears on Common Paths?

Table IV reports five stable metrics from the same-daemon lmbench experiment. Each entry is the mean of three runs, and every NACRE mean is within 3.5% of the corresponding baseline mean. We omit process, Unix-socket, file-syscall, and memory-bandwidth cases whose records contain missing values or large outliers.

The three repetitions are insufficient to interpret small apparent speedups, and the pipe-latency samples are mixed. The supported conclusion is narrower: the retained syscall and pipe means show no large software-path regression in this QEMU experiment. These aggregate measurements do not attribute cost to `acall/aret`, PFN-role checks, protected PTE commits, or TLB effects.

D. RQ3: Can the Prototype Run Service Workloads?

For nginx, we transferred 1 GiB at concurrency 16 for each of eight static object sizes from 4 KiB to 512 KiB, with three runs per size and arm. All 48 runs completed with zero failed requests. Across the eight per-size means, the NACRE/origin throughput ratio ranges from 0.950 to 1.004. The ratio of the equally weighted means is 0.9813 (16,557.46 versus 16,873.20 KB/s). Thus, this QEMU matrix shows that nginx completes while its measured aggregate throughput is 1.87% below the baseline; it does not provide a confidence interval over independent experiment matrices.

Three additional workloads provide functional sanity evidence. Five netperf configurations completed with zero return codes. SQLite completed the same SQL workload in both arms and produced identical output hashes. A memcached probe completed a direct `set/get` and a one-request memtier invocation. Because these workloads lack repeated, controlled comparative measurements, we use them only to demonstrate execution of the tested paths.

E. RQ4: What Remains Open?

Hackbench identifies a scalability boundary rather than a benchmark result. With 2,000 workers and five loops, both arms returned zero, but the NACRE run took $1.50\times$ longer according to Hackbench’s internal timer and $4.59\times$ longer end to end. Most additional time precedes the internal timer, suggesting—but not establishing—a cost in worker creation, registration, or readiness. A larger NACRE run reached `CONTAINER_FULL` and failed to attach a child. These single-run observations expose capacity and lifecycle work; they do not establish steady-state scalability.

The current security evidence is likewise positive-path evidence. The broad regression completed without falling back to raw S-mode access to `PRIV` pages, and the dedicated fork test exercised the final monitor-owned copy-and-install path. We have not run negative tests for wrong owner or `cid`, stale task bindings, role or permission changes, failed `PEND` transactions, or clear-before-reuse. Cross-container sharing, multicore agent entry, and attestation also remain unevaluated. The evaluation therefore supports the feasibility of the exercised single-container paths, not a complete malicious-host security or compatibility claim.

VIII. DISCUSSION AND LIMITATIONS

Prototype scope. The current artifact consists of the core NACRE software stack and a QEMU model of its architectural mechanisms. It exercises protected private-memory transitions, monitor-mediated page-table commits, agent-directed trap delegation, and the main single-container `fork/COW/exit` paths. The results in this paper therefore establish feasibility only for these modeled paths; they do not validate the target design’s complete multi-container, sharing, multicore, or attestation semantics.

Architectural realization and sPMP. Region mechanisms such as PMP or sPMP can deny ordinary S-mode accesses to reserved monitor and agent memory. They cannot distinguish containers within host-managed memory, bind translations to a container, or deliver a protected trap to a peer-privilege agent. NACRE therefore uses region protection as a bootstrap boundary, then adds architectural container-identity state, identity-aware translation checks, and identity-aware event delivery. The QEMU model captures only the functional semantics of these mechanisms.

Portability. The design requires an unforgeable domain identity, protected ownership metadata, a commit path that the host cannot bypass, and protected event delivery. Arm or x86 mechanisms could approximate parts of this interface, but placing the agent in a separate VM, realm, or secure-world context may reintroduce address-space switches, additional trusted software, or extra privilege transitions. We do not evaluate the security or performance of such ports.

Untrusted I/O and trusted paths. NACRE deliberately leaves Linux and its device drivers outside the TCB. Even when trusted code bounds a delegated buffer and sanitizes architectural return state, it cannot make a malicious filesystem, network stack, or device available or correct. This limitation follows from treating the host system-call interface as an untrusted service boundary [10]. Confidential storage and network channels therefore require authenticated encryption to a trusted endpoint, while direct device assignment additionally requires DMA isolation and a device-specific reset and ownership protocol. A verifiable user path could provide stronger assurance for selected devices, but is orthogonal to the CPU execution boundary studied here [49].

Table V: Evidence boundary of the current QEMU prototype.

Claim	Current evidence	Supported scope	Evidence still required
Linux lifecycle paths	Prior-revision eight-path regression; implemented-fork dedicated test	Honest execution of the covered fault, uaccess, fork, exec, and teardown paths	Broad regression on one final revision and a wider compatibility suite
Common-path cost	Three-run same-daemon lmbench subset	Five QEMU syscall and pipe means within 3.5%	More repetitions, distributions, and mechanism-level decomposition
Service execution	Repeated nginx matrix; netperf, SQLite, and memcached functional checks	Completion of the tested service paths and nginx throughput in one QEMU matrix	Controlled repeated application experiments
Protected transitions	Strict positive-path regression and final fork copy/install test	Exercised single-container transitions	Wrong-domain, stale-state, failed-transaction, and reuse negative tests

IX. RELATED WORK

Enclaves and shielded applications. Enclave systems protect an application address space from an untrusted OS. Haven, SCONE, Graphene-SGX, Panoply, SGX-LKL, and Occlum recover different subsets of OS functionality through library operating systems, shielded runtimes, or carefully mediated host calls [5], [6], [9], [34], [39], [42]. Keystone and Penglai provide extensible or scalable enclave abstractions on RISC-V [17], [26]. These systems establish strong application-centric protection, but multi-process and resource-sharing semantics must be reconstructed inside or coordinated across enclave boundaries. NACRE instead makes the Linux container process group the target protection and lifecycle unit while retaining host-native process and memory management.

Confidential virtual machines. AMD SEV-SNP, Intel TDX, and Arm CCA protect guest execution from a hostile hypervisor or host [4], [21], [38]. Kata Containers and Confidential Containers use this VM boundary to protect containerized workloads with a guest kernel and supporting services inside the trusted domain [14], [15], [35]. Firecracker reduces the footprint and startup cost of this virtualization model for serverless workloads [1]. These approaches offer mature Linux semantics within the guest, whereas NACRE explores a container-granularity boundary that lets host Linux retain scheduling and resource management without placing it in the security TCB.

Container-granularity protection. TZ-Container, BlackBox, ConMonitor, RContainer, and Fasco protect containers from an untrusted host using combinations of TrustZone, virtualization, container monitors, and confidential-computing extensions [20], [43], [44], [46], [48]. Arca and ACE further explore lightweight confidential execution for cloud-native or embedded settings [28], [33]. These systems differ in their trusted service layer, protection context, and division of lifecycle responsibility. NACRE occupies another point in this design space: it introduces a hardware-recognized container identity and routes protected events to an isolated peer-privilege agent, while a higher-privilege monitor commits security-sensitive ownership and mapping transitions.

Protection over an untrusted commodity OS. Overshadow and InkTag retrofit protected application execution over an un-

trusted commodity OS using virtualization and authenticated or trusted memory-management interfaces [11], [19]. Dune exposes privileged CPU facilities to applications, illustrating how hardware virtualization can support protection mechanisms outside the kernel without replacing the kernel’s service role [7]. Iago attacks show that protecting memory alone is insufficient when an adversarial OS can manipulate system-call results [10]. NACRE shares the principle that the host may provide policy and services but must not authorize protected transitions; it specializes that separation to native container identity, trap delivery, and Linux lifecycle operations.

Paravirtualized secure containers. ParaCell uses memory protection keys (MPK) for intra-address-space isolation and intent-driven memory management to reduce user/kernel transition cost and improve memory elasticity for paravirtualized secure containers [45]. Its host Linux acts as the virtual-machine monitor and remains responsible for container scheduling, interrupt forwarding, device I/O, and host mappings. ParaCell therefore addresses efficiency under a trusted-host model, whereas NACRE separates host resource management from security authority under a compromised host.

Elastic and confidential serverless execution. Reusable enclaves, plug-in enclaves, and Split Containers reduce initialization or memory cost for confidential serverless functions [27], [40], [47]. Other systems accelerate function creation through optimized containers, snapshots, or language/runtime isolation [2], [3], [8], [16], [30], [41]. These techniques are complementary to NACRE’s focus: they optimize how an execution environment is created or reused, while NACRE asks how a dynamic Linux container can remain the hardware-recognized protection unit when the host kernel is untrusted.

X. CONCLUSION

We presented NACRE, which makes a Linux container process group the protection unit while leaving scheduling and resource management to an untrusted host. A hardware-recognized container identity and an isolated S-mode agent keep frequent interactions on the native Linux path, while an M-mode monitor commits protected mappings and lifecycle transitions. Our QEMU prototype exercises the core single-container private-memory and trap paths, including exec,

demand paging, fork, COW, and teardown. In the recorded experiments, the three-run means of five Imbench syscall and pipe metrics were within 3.5% of the same-daemon runc-origin baseline, and the NACRE/baseline ratio of equally weighted nginx mean throughputs across eight object sizes was 0.981. These results support the feasibility of separating host resource management from security authority on the modeled data and control paths. The cooperative identity and lifecycle checks that remain in the prototype delimit, rather than complete, its malicious-host security claim.

REFERENCES

- [1] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, 2020, pp. 419–434.
- [2] I. E. Akkus, R. Chen, I. Rimac, M. Stein, K. Satzke, A. Beck, P. Aditya, and V. Hilt, “SAND: Towards high-performance serverless computing,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, 2018, pp. 923–935.
- [3] L. Ao, L. Izhikevich, G. M. Voelker, and G. Porter, “FaaSnap: FaaS made fast using snapshot-based VMs,” in *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys)*, 2022, pp. 730–746.
- [4] Arm, “Arm Confidential Compute Architecture,” <https://www.arm.com/architecture/security-features/arm-confidential-compute-architecture>, 2023.
- [5] S. Arnaudov, B. Trach, F. Gregor, T. Knauth, A. Martin, C. Priebe, J. Lind, D. Muthukumar, D. O’keefe, M. L. Stillwell *et al.*, “SCONE: Secure linux containers with Intel SGX,” in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 2016, pp. 689–703.
- [6] A. Baumann, M. Peinado, and G. Hunt, “Shielding applications from an untrusted cloud with haven,” in *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, 2014, pp. 267–283.
- [7] A. Belay, A. Bittau, A. Mashtizadeh, D. Terei, D. Mazières, and C. Kozyrakis, “Dune: Safe user-level access to privileged CPU features,” in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI 12)*. USENIX Association, 2012, pp. 335–348.
- [8] J. Cadden, T. Unger, Y. Awad, H. Dong, O. Krieger, and J. Appavoo, “SEUSS: Skip redundant paths to make serverless fast,” in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020, pp. 32:1–32:15.
- [9] C. che Tsai, D. E. Porter, and M. Vij, “Graphene-SGX: A practical library OS for unmodified applications on SGX,” in *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. Santa Clara, CA: USENIX Association, Jul. 2017, pp. 645–658. [Online]. Available: <https://www.usenix.org/conference/atc17/technical-sessions/presentation/tsai>
- [10] S. Checkoway and H. Shacham, “Iago attacks: Why the system call API is a bad untrusted RPC interface,” in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013, pp. 253–264.
- [11] X. Chen, T. Garfinkel, E. C. Lewis, P. Subrahmanyam, C. A. Waldspurger, D. Boneh, J. Dwoskin, and D. R. K. Ports, “Overshadow: A virtualization-based approach to retrofitting protection in commodity operating systems,” in *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2008, pp. 2–13.
- [12] Cloud Native Computing Foundation, “Kubernetes at the Edge: Organizations Are Using Edge Technologies, but There Is Room to Grow,” https://www.cncf.io/wp-content/uploads/2021/05/KubernetesEdge_Survey_Report_2021_v2.pdf, 2021, accessed: 2026-07-27.
- [13] —, “The CNCF Annual Cloud Native Survey: The Infrastructure of AI’s Future,” <https://www.cncf.io/reports/the-cncf-annual-cloud-native-survey/>, 2026, accessed: 2026-07-27.
- [14] Confidential Containers Community, “Confidential containers,” <https://confidentialcontainers.org/>, 2024.
- [15] —, “Design Overview for Confidential Containers,” <https://confidentialcontainers.org/docs/architecture/design-overview/>, 2026, accessed: 2026-07-27.
- [16] D. Du, T. Yu, Y. Xia, B. Zang, G. Yan, C. Qin, Q. Wu, and H. Chen, “Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting,” in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020, pp. 467–481.
- [17] E. Feng, X. Lu, D. Du, B. Yang, X. Jiang, Y. Xia, B. Zang, and H. Chen, “Scalable memory protection in the PENGDAI enclave,” in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 2021, pp. 275–294.
- [18] A. Ghiti, “Virtual Memory Layout on RISC-V Linux,” <https://docs.kernel.org/arch/riscv/vm-layout.html>, 2021, accessed: 2026-06-21.
- [19] O. S. Hofmann, S. Kim, A. M. Dunn, M. Z. Lee, and E. Witchel, “InkTag: Secure applications on an untrusted operating system,” in *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013, pp. 265–278.
- [20] Z. Hua, Y. Yu, J. Gu, Y. Xia, H. Chen, and B. Zang, “TZ-Container: Protecting container from untrusted OS with ARM TrustZone,” *Science China Information Sciences*, 2021.
- [21] Intel, “Intel Trust Domain Extensions,” <https://software.intel.com/content/dam/develop/external/us/en/documents/tdxwhitepaper-v4.pdf>, 2020.
- [22] Intel, “Intel 64 and IA-32 Architectures Software Developer’s Manual,” <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>, 2026, accessed: 2026-06-23.
- [23] D. Kaplan, J. Powell, and T. Woller, “Amd memory encryption,” *White paper*, 2016.
- [24] G. M. Kurtzer, V. Sochat, and M. W. Bauer, “Singularity: Scientific containers for mobility of compute,” *PLOS ONE*, vol. 12, no. 5, p. e0177459, 2017.
- [25] LangChain, “Self-host Standalone Agent Servers,” <https://docs.langchain.com/langsmith/deploy-standalone-server>, 2026, accessed: 2026-07-27.
- [26] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanović, and D. Song, “Keystone: An open framework for architecting trusted execution environments,” in *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020.
- [27] M. Li, Y. Xia, and H. Chen, “Confidential serverless made efficient with plug-in enclaves,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 306–318.
- [28] D. Lu, M. Sun, Q. Zhang, Y. Liu, J. Zhang, X. Dong, Y. Shen, and J. Ma, “Arca: A lightweight confidential container architecture for cloud-native environments,” arXiv preprint arXiv:2601.01214, 2026. [Online]. Available: <https://arxiv.org/abs/2601.01214>
- [29] D. Merkel, “Docker: Lightweight linux containers for consistent development and deployment,” *Linux Journal*, vol. 2014, no. 239, 2014.
- [30] E. Oakes, L. Yang, D. Zhou, K. Houck, T. Harter, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “SOCK: Rapid task provisioning with serverless-optimized containers,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. USENIX Association, 2018, pp. 57–70.
- [31] Open Container Initiative, “Open Container Initiative Runtime Specification: Linux,” <https://specs.opencontainers.org/runtime-spec/config-linux/?v=v1.3.0>, 2025, accessed: 2026-07-27.
- [32] OpenHands, “Docker Sandbox,” <https://docs.openhands.dev/openhands/usage/sandboxes/docker>, 2026, accessed: 2026-07-27.
- [33] W. Ozga, G. D. H. Hunt, M. V. Le, L. Gaher, A. Shinnar, E. R. Palmer, H. Jamjoom, and S. Dragone, “ACE: Confidential computing for embedded RISC-V systems,” arXiv preprint arXiv:2505.12995, 2025.
- [34] C. Priebe, D. Muthukumar, J. Lind, H. Zhu, S. Cui, V. A. Sartakov, and P. Pietzuch, “SGX-LKL: Securing the host OS interface for trusted execution,” arXiv preprint arXiv:1908.11143, 2019.
- [35] A. Randazzo and I. Tinnirello, “Kata containers: An emerging architecture for enabling mec services in fast and secure way,” in *2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*. IEEE, 2019, pp. 209–214.
- [36] RISC-V International, “RISC-V instruction set manual, volume ii: Privileged architecture,” <https://docs.riscv.org/reference/isa/priv/priv-ind-ex.html>, 2024.
- [37] —, “RISC-V supervisor binary interface specification,” <https://github.com/riscv-non-isa/riscv-sbi-doc>, 2024.
- [38] A. SEV-SNP, “Strengthening vm isolation with integrity protection and more,” *White Paper*, January, 2020.

- [39] Y. Shen, H. Tian, Y. Chen, K. Chen, R. Wang, Y. Xu, Y. Xia, and S. Yan, "Occlum: Secure and efficient multitasking inside a single enclave of Intel SGX," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2020, pp. 955–970.
- [40] J. Shi, J. Gu, Y. Xia, and H. Chen, "Serverless functions made confidential and efficient with split containers," in *34th USENIX security symposium (USENIX Security 25)*, 2025.
- [41] S. Shillaker and P. Pietzuch, "Faasm: Lightweight isolation for efficient stateful serverless computing," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*. USENIX Association, 2020, pp. 419–433.
- [42] S. Shinde, D. Le Tien, S. Tople, and P. Saxena, "Panoply: Low-TCB linux applications with SGX enclaves," in *Network and Distributed System Security Symposium (NDSS)*, 2017.
- [43] L. Song, Y. Zhang, F. Zhang, Y. Ding, B. Zhou, J. Yu, and Y. Tan, "Shielded but lightweight: Building practical confidential containers with ARM CCA," arXiv preprint arXiv:2605.26018, 2026.
- [44] A. Van't Hof and J. Nieh, "BlackBox: a container security monitor for protecting containers on untrusted operating systems," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 683–700.
- [45] Y. Wu, X. Wang, J. Gu, and H. Chen, "ParaCell: Paravirtualized secure containers with lightweight intra-container isolation and intent-driven memory management," arXiv preprint arXiv:2605.20906, 2026. [Online]. Available: <https://arxiv.org/abs/2605.20906>
- [46] S. Xu, Q. Zhou, Z. Zhang, X. Jia, D. Liu, H. Huang, H. Du, and Z. Song, "Conmonitor: Lightweight container protection with virtualization and vm functions," in *Proceedings of the 2024 ACM Symposium on Cloud Computing*, 2024, pp. 755–773.
- [47] S. Zhao, P. Xu, G. Chen, M. Zhang, Y. Zhang, and Z. Lin, "Reusable enclaves for confidential serverless computing," in *32nd USENIX security symposium (USENIX Security 23)*, 2023, pp. 4015–4032.
- [48] Q. Zhou, W. Cao, X. Jia, P. Liu, S. Zhang, J. Chen, S. Xu, and Z. Song, "Rcontainer: A secure container architecture through extending arm cca hardware primitives," in *Network and Distributed System Security Symposium (NDSS)*. NDSS, 2025.
- [49] Z. Zhou, V. D. Gligor, J. Newsome, and J. M. McCune, "Building verifiable trusted path on commodity x86 computers," in *2012 IEEE symposium on security and privacy*. IEEE, 2012, pp. 616–630.